

Resumo Executivo

Este relatório propõe uma dinâmica de **2 horas** para ensino de *code smells* e refatoração em um ambiente colaborativo usando VS Code, com cálculo numérico em C++ exposto via Python. A atividade terá como foco a integração eficiente entre **C++ (núcleo matemático)** e **Python (camada de uso)**, utilizando bibliotecas modernas de ligação (como *pybind11* ou *nanobind*) e analisadores de qualidade de código (como *clang-tidy*, *Ruff* e *CodeScene*). O objetivo é que os alunos aprendam a identificar problemas de design (mau cheiro, complexidade, baixo acoplamento, etc.) e a aplicar refatorações práticas sem quebrar o comportamento, medindo melhorias em métricas de qualidade (p. ex. *Code Health* do CodeScene). Usaremos uma abordagem mista: metade do tempo com equipes trabalhando isoladamente em seus códigos, e metade com o professor coordenando e usando Live Share para apresentar resultados em tempo real.

Principais pontos do projeto:

- **Objetivos de aprendizagem:** detecção e correção de *code smells*; integração C++/Python via módulos nativos; uso de ferramentas de análise estática em tempo real; colaboração remota.
- **Público-alvo:** alunos com conhecimentos básicos de Python e C++, idealmente sem necessidade de experiência prévia com ferramentas específicas de qualidade de código.
- **Pré-requisitos:** máquina host Linux/macOS com VS Code, Python 3.10+, compilador C++ (C++17), pip, e acesso à internet para extensões.
- **Métricas de sucesso:** melhorias no *Code Health* (ferramenta CodeScene) e redução de warnings estáticos, mantendo todos os testes automatizados verdes (pytest). Também avaliamos engajamento (perguntas, discussões) e cumprimento dos cronogramas.

Neste documento discutimos as **abordagens técnicas** (ligação direta FFI versus comunicação IPC), com comparativos *pybind11/nanobind/cppyy/ctypes* e FFI versus *gRPC/ZeroMQ*. Propomos a **stack de ferramentas** recomendada (CMake + scikit-build-core, Eigen, pybind11, pytest, clangd/clang-tidy, Ruff, CodeScene, Live Share) e justificamos cada escolha com base em performance e facilidade de uso. Apresentamos um **roteiro minuto a minuto** da atividade, papéis de professor e alunos, checkpoints e tarefas. Também detalhamos a **configuração do host** (instalações, configurações VS Code, scripts de build) e instruções mínimas para os alunos (especialmente para o uso do Live Share). Finalmente, fornecemos exemplos de código (um solver linear em C++ com *Eigen*, binding Python e testes *pytest*) contendo *smells* artificiais, seguidos de exercícios de refatoração com critérios de aceitação claros.

Todo o material didático (.md) e comandos práticos necessários são apresentados de forma explícita neste relatório, permitindo salvar e reutilizar diretamente os arquivos de configuração (`README.md`, `HOST_SETUP.md`, `STUDENT_GUIDE.md`, `EXERCISES.md`). Para a visualização de métricas em tempo real, usamos o **CodeScene** (monitor *Code Health*) integrado ao editor. O VS Code Live Share (mesmo em modo manutenção) é usado para a colaboração e apresentação síncrona. Por fim, incluímos diagramas *Mermaid* do cronograma e da arquitetura de integração (Python $\rightleftarrows$ C++), além de muitas referências oficiais que embasam cada escolha tecnológica.

Objetivos de Aprendizagem

Os objetivos da dinâmica são:

- **Identificar *code smells* e dívida técnica:** reconhecer problemas comuns em código (métodos longos, classes gigantes, *feature envy*, *primitive obsession* etc.) tanto em Python quanto em C++.
- **Aprender a refatorar com segurança:** aplicar transformações de código que preservam o comportamento (como extrair método, renomear variáveis, reestruturar condicionais) e comprovar com testes.
- **Entender integração C++/Python:** saber expor funções e classes de C++ para Python usando módulos nativos (*pybind11*/*nanobind*) ou, como comparação, comunicar via IPC (ZeroMQ/gRPC).
- **Usar ferramentas de análise estática em tempo real:** ver de forma interativa *code smells*, avisos de compilador e linter (*clang-tidy*, *Ruff*) enquanto edita, aprendendo a agir antes de cometer bugs.
- **Colaboração em tempo real:** praticar programação em par/grupo com VS Code Live Share, organizando turnos de edição e discussão sob supervisão.

Público-Alvo

Profissionais ou alunos de **Engenharia de Software** ou áreas afins, com conhecimentos *básicos/intermediários* de programação em **Python e C++**. Não se espera familiaridade prévia com integração de linguagens ou ferramentas específicas de código estático. Ideal para *bootcamps*, cursos de graduação ou treinamentos corporativos de qualidade de código.

Pré-requisitos

- **Plataforma:** host Linux ou macOS (Windows é possível, mas foco em UNIX-like).
- **Software:** VS Code (última versão), Python 3.10+, extensão *Visual Studio Live Share*, compilador C++ (*g++/clang++*), *pip*.
- **Instalações no host (ver seção de configuração):** *clangd*, *clang-tidy*, *Ruff*, *CodeScene VS Code extension*, *Eigen* (biblioteca C++), *pybind11/nanobind* (header-only), *scikit-build-core* (*pip*), *pytest*, etc.
- **Ambiente de desenvolvimento:** conhecimento básico de Git e VS Code, acesso ao repositório do exercício.

Se adotado modelo *colaborativo host-alunos* (via Live Share), os alunos **precisam apenas do VS Code e da extensão Live Share instalada**; o host terá as ferramentas de análise. Se cada aluno trabalhar em seu próprio ambiente, será preciso instalar *clangd*, *Ruff* e afins em todas as máquinas.

Métricas de Sucesso

Mediremos o sucesso da atividade por meio de:

- **Qualidade do código:** comparação do *Code Health* antes/depois (ferramenta *CodeScene* pontua de 0 a 10). Meta: aumentar o *Code Health* geral dos arquivos críticos.
- **Linhas de *smells*:** redução de avisos de linter/compilador (*clang-tidy*, *Ruff*). Menos *warnings* indicam código mais limpo.

- **Testes automatizados:** todas as suítes *pytest* devem permanecer verdes após refatorações. Critério: refatorações válidas não quebram testes.
- **Feedback qualitativo:** discussão em sala, observação de participação ativa (perguntas, sugestões), e capacidade de explicar escolhas de refatoração.
- **Cumprimento do cronograma:** as atividades propostas devem ser completadas dentro dos 120 minutos planejados (ajuste dinâmico se necessário).

Comparação de Abordagens

FFI: pybind11 vs nanobind vs cppyy vs ctypes

Abordagem	Descrição	Compilação	Sobrecarga em Tempo de Execução	Facilidade de Uso	Uso Recomendado
pybind11	Biblioteca moderna em C++ para expor funções/classes C++ em Python 【7†L59-L64】 . Header-only, sem dependências externas além do CPython.	Requer CMake/compilação durante o <i>build</i> do pacote.	~ overhead de chamada C++ normal (baixo) 【4†L108-L116】 .	Sintaxe clara (parecida com Boost.Python, porém mais enxuta) 【7†L59-L64】 .	Integração profunda in-process. Boa para C++ com STL/Eigen.
nanobind	Successor de pybind11 pelo mesmo autor. Mais rápido: binários ~5× menores, ~4× compilação mais veloz 【4†L58-L66】 . Vantagens de C++17 (ex.: smart pointers internamente).	Mesmo modelo do pybind11 (CMake + compilação).	Menor overhead que pybind11 (~10× melhor em chamadas de classe) 【4†L106-L114】 .	API quase idêntica ao pybind11; migração simples. Suporta Stable ABI (Python 3.12+) 【23†L82-L90】 .	Recomendado se compilar os bindings: mais leve que pybind11.

Abordagem	Descrição	Compilação	Sobrecarga em Tempo de Execução	Facilidade de Uso	Uso Recomendado
cppyy	Ligação <i>JIT</i> dinâmica via Cling/LLVM (parte do projeto CERN ROOT) [4†L130-L138] . É possível carregar binários pré-compilados (<code>load_library</code>) e chamar nomes C++.	Não requer compilação de wrapper (gera via <i>JIT</i> em runtime). Precisa de Clang/LLVM/Cling presentes.	Chamadas têm overhead (JIT + conversão de dados) e dependem do Clang runtime [4†L130-L138] .	Muito fácil para prototipagem e exploração (carrega headers C++ diretamente no Python). Menos usado em produção.	Bom para explorar binários legados rapidamente. Não gera módulo redistribuível.
ctypes	Biblioteca nativa do Python para chamar C (não suporta C++ diretamente). Requer colocar <code>extern "C"</code> nas funções.	Geração de <i>DLLs/.so</i> com interface C (pode usar compilador C++ mas exportando funções em <code>extern "C"</code> [13†L325-L332]).	Chamada leve (C ABI), mas sem verificação de tipo além do que você declara manualmente.	Verboso: obriga declarar manualmente assinaturas (parâmetros e tipos), não suporta classes C++/exceções [13†L329-L332] .	Útil só se a biblioteca C++ já fornecer C extern API; caso contrário, exige wrapper em C manual.

Pybind11 e Nanobind fornecem bindings *in-process* completos, permitindo chamadas diretas de C++ no Python com interoperabilidade (por exemplo, conversão de `Eigen::MatrixXd` para `numpy.ndarray` sem cópia [7†L137-L140]). Ctypes, em contraste, só suporta interface C (precisa envolver o C++). O cppyy é interessante para *exploração interativa* (carrega *live* código C++), mas **não gera pacote binário pré-compilado** (as binding são geradas em tempo de execução) [4†L132-L140] .

IPC: gRPC vs ZeroMQ (vs FFI)

Abordagem	Descrição	Desempenho/ Latência	Complexidade de Implementação	Escopo de Uso	Exemplo de Ferramenta
FFI (compartilhado)	Integra C++ e Python no mesmo processo via módulo <i>shared object</i> . Alta taxa de chamadas; evita serialização.	Muito alta (baixa latência), sem overhead de rede.	Requer compilação de bindings, mas simples no uso (uma chamada de função).	Quando desempenho é crítico e não precisa desacoplamento de processos. Ideal para cálculos pesados embutidos em apps Python.	Pybind11/Nanobind (descrito acima).
gRPC	Framework de RPC (Protocol Buffers + HTTP/2). Define interfaces via <code>.proto</code> . Gere stubs Python/C++.	Alto (baseado em HTTP/2, streaming, TLS). Mas introduz overhead de serialização/deserialização.	Mais complexo: define <code>.proto</code> , compila <i>protos</i> , código de servidor/cliente.	Sistemas distribuídos ou microserviços, comunicação inter-processos de forma robusta.	gRPC – “RPC framework de alto desempenho” [20†L17-L22] .
ZeroMQ	Biblioteca de mensageria (REQ/REP, PUB/SUB etc.) <i>message queue</i> leve [16†L47-L50] .	Muito alta (overhead de rede reduzido, suporte a IPC local e TCP) [16†L47-L50] . Suporta padrões assíncronos.	Baixo a médio: estilo socket, mas abstrai conexões. Precisa definir <i>protocolos</i> de mensagens (ex.: JSON).	Quando precisa de comunicação flexível, escalável e agnóstica de linguagem. Uso comum: sistemas de mensageria em tempo real.	ZeroMQ – mensageria leve entre C++ e Python.

Em resumo, **FFI** (módulo nativo) fornece alto desempenho e simplicidade para cálculos matemáticos embutidos, mas amarra a comunicação aos limites de uma única máquina/processo. Já **IPC** (gRPC, ZeroMQ) permite desacoplamento (cada linguagem pode rodar em processo separado ou até máquinas distintas), mas requer serialização de dados e maior esforço de infraestrutura. Por exemplo, gRPC é ideal para RPC estruturado (escolha para sistemas em microserviços), enquanto ZeroMQ permite padrões de mensageria customizados (apesar de não gerenciar protobufs nativamente, contrabalanceando isso com flexibilidade) [17†L99-L102] [16†L47-L50] .

Neste workshop, recomendamos **FFI (pybind11/nanobind)** para a parte de integração principal (cálculos matemáticos em C++ no Python), já que o foco é desempenho e aprendizado de bindings nativos. No entanto, podemos mencionar IPC como alternativa teórica: por exemplo, se o “código binário C++” fosse um serviço isolado, ZeroMQ ou gRPC permitiriam conectar Python via rede/in-memory. Em particular, ZeroMQ proporciona *IPCs* de baixa latência e comunicação multi-linguagem [【16†L47-L50】](#) . Porém, a complexidade extra de definir um protocolo gRPC (ProtoBufs) ou lidar com padrões ZeroMQ é além do escopo de um exercício de refatoração focado em *code smells*.

Stack Recomendado

Para a atividade prática, a pilha de ferramentas escolhida é baseada na simplicidade de uso, compatibilidade com C++/Python e suporte a extensões do VS Code:

- **C++/Build: CMake & scikit-build-core** – CMake é padrão para projetos C++. Usamos [scikit-build-core](#) para integrar o processo de build CMake ao `pip install .` Ele permite criar um pacote Python com extensões em CMake (tem exemplos prontos para pybind11/nanobind) [【23†L95-L102】](#) . Por exemplo, `scikit-build init --backend pybind11` gera um template já configurado [【23†L95-L102】](#) .
- **Álgebra Linear: Eigen** – Biblioteca C++ robusta para vetores e matrizes, otimizada. Permite operações numéricas sem cópias extras (com pybind11, buffers NumPy mapeados direto para `Eigen::MatrixXd` [【7†L137-L140】](#)). A [documentação oficial](#) destaca o uso em álgebra linear.
- **Bindings C++↔Python: pybind11 ou nanobind** – Ferramentas para expor o código C++ ao Python. Pybind11 é estável e popular [【7†L59-L64】](#) ; nanobind é similar e traz ganhos de performance no build/runtime [【4†L58-L66】](#) . Nos exemplos, usaremos pybind11 por ser mais conhecido, mas o processo é quase idêntico em nanobind.
- **Linters/Analizadores estáticos: clangd/clang-tidy (C++), Ruff (Python)** –
 - *clang-tidy* é um “linter” C++ baseado no Clang [【26†L32-L36】](#) . Detecta padrões problemáticos, estilo e potenciais bugs. Pode aplicar *fixes* automáticos (flag `-fix`).
 - *clangd* (plugin VSCode) roda em background e já incorpora checks do *clang-tidy* em tempo real.
 - *Ruff* é um linter Python extremamente rápido (0,01–0,1s em um projeto grande) [【29†L57-L60】](#) e integra Flake8, Black, pyupgrade, etc. em um só pacote. Ele oferece *auto-fix* de muitos avisos e feedback imediato.
- **CodeScene IDE (VS Code extension)** – Fornece um indicador visual de **Code Health** e sinaliza *code smells* conforme se digita [【36†L225-L234】](#) . Suporta análise em Python e C++ (mais de 25 smells detectados). Também mostra uma pontuação agregada de qualidade do arquivo atual. Usaremos para medir ganhos de refatoração.
- **Live Share (VS Code extension)** – Permite colaboração em tempo real: vários alunos e o professor editam o mesmo projeto. Apesar de estar em modo manutenção (sem novos recursos planejados) [【31†L41-L48】](#) , mantém a funcionalidade principal. O host (professor) executa as ferramentas; alunos apenas precisam entrar na sessão.

Essas escolhas são justificadas porque:

- **Consistência e fácil setup:** usar scikit-build-core com CMake padroniza a geração do módulo Python e evita scripts manuais. O `CMakeLists.txt` pode ser mínimo, apontando fontes C++ e ligando Eigen. O `pyproject.toml` com scikit-build simplifica o `pip install -e .` .

- **Desenvolvimento ágil:** clangd e Ruff fornecem feedback instantâneo no editor sobre issues menores (ex.: uso de ponteiros, estilos, importações faltantes). Assim, o aluno “vê” problemas de maintainability sem sair do VS Code. O *autofix* do Ruff e *clang-tidy* aceleram a correção.
- **Medição objetiva:** usar CodeScene (gratuito para pequenos projetos) introduz um indicador quantitativo de qualidade. Cada refatoração pode mostrar como a pontuação (Code Health) melhorou. Isso reforça o aprendizado “*forma muda, comportamento não*” de refatoração.
- **Experiência didática:** combinar ferramentas automatizadas com revisão humana. Por exemplo, o professor pode destacar manualmente um *God Object* e depois mostrar como o CodeScene o marca. A colaboração via Live Share facilita demonstrações síncronas sem complicar instalações em cada máquina.

Em resumo, o *stack* recomendado é:

```
Ubuntu/Mac Host: VS Code + clangd + clang-tidy + Ruff + CodeScene IDE ext + Live
Share ext +
Python 3.10+ + pip scikit-build-core, pybind11 + Eigen + pytest + pytest-cov.
```

Cada item tem documentação ativa e ampla comunidade (além das citações acima).

Roteiro da Atividade (2h)

A sessão será dividida em etapas intercalando explicações do professor e momentos hands-on das equipes. Os papéis principais são:

- **Professor/Host:** guia a atividade, prepara o ambiente, apresenta conceitos e coleta resultados. Controla a sessão do Live Share.
- **Alunos individuais ou equipes:** participam ativamente, analisando código, propondo refatorações, executando tarefas e apresentando resultados.

Segue o cronograma minuto a minuto (os tempos são aproximados):

```
timeline
    title Cronograma da Dinâmica (2h)
    00:00 : Início – Boas-vindas e objetivos da aula
    00:05 : Apresentar código exemplar (Python e C++) com *code smells*
    00:15 : Discussão aberta: “O que há de ruim neste código?”
    00:20 : Demonstração das ferramentas (CodeScene, clangd, Ruff) no VS Code
    00:30 : Divisão em equipes / definição de papéis
    00:35 : Tarefa 1: Refatoração inicial de parte do código (15 min)
    00:50 : **Checkpoint 1** – Apresentação de avanços (Code Health, warnings)
    00:55 : Tarefa 2: Refatoração adicional (15 min, invertendo papéis nos
times)
    01:10 : **Checkpoint 2** – Comparação de resultados finais (melhorias de
métricas)
    01:15 : Live Share demo: professor faz exemplo de refactoring e executa
tasks
```

01:25 : Perguntas & Respostas sobre metodologias de refatoração
01:35 : Exemplos de CLI de compilação/testes e uso de Tasks do VS Code
01:45 : Feedback geral das equipes (o que melhorou / desafios)
01:50 : Próximos passos e recursos para estudo adicional
02:00 : Encerramento

- **Papel dos estudantes:** inicialmente, os alunos analisam individualmente o código ruim (em Python e C++), identificam possíveis *smells* e discutem em grupo. No cronograma, vemos que após cada *checkpoint* eles trocam de foco (por exemplo, quem trabalhou no Python passa para C++ na Tarefa 2), garantindo exposição a ambos os lados da integração.
- **Checkpoints:** momentos em que cada time mostra brevemente suas refatorações. Isso reforça o espírito colaborativo e permite ao professor avaliar métricas de sucesso (Code Health, *warnings*, status dos testes).
- **Uso do Live Share:** Durante a atividade, o professor hospeda a sessão. Nos primeiros momentos, apenas demonstra como as extensões sinalizam problemas (e.g., o CodeScene mostra “Complex Method” ou “Deep Nesting” no menu Problems). Em seguida, cada time opera em sua própria instância (branches separadas) de forma independente. Ao final, o professor pode compartilhar exemplos de correção via Live Share (por exemplo, um aluno coloca o cursor no código e professor mostra a correção via *shared terminal* ou *pair programming*).
- **Check do Cronograma:** O tempo de refatoração (30 min no total) pode ser ajustado, mas deve incluir metas claras (por ex. “aumentar o Code Health de X para Y” ou “remover 5 warnings de clang-tidy”). Os últimos 15 minutos reservam a revisão geral, demonstração de tarefas do VS Code (como rodar todas as análises com uma Task) e referências adicionais.

O cronograma visual acima serve como guia. Se necessário, ajuste o tempo (p. ex., alongar revisões se a turma tiver dúvidas). O importante é manter o foco: *analisar código* → *planejar refatoração* → *aplicar* → *validar com tests e ferramentas* → *iterar*.

Configuração do Host

Instalações no Host

No computador do instrutor (host), instale:

- **VS Code** (última versão).
- **Extensões VS Code:** Live Share (host), CodeScene IDE, Python (Microsoft), C/C++ (Microsoft) com clangd, Ruff (astral-sh). Crie um *Profile* do VS Code para a aula contendo essas extensões e configurações sugeridas.
- **Compilador C++:** g++ ou clang++ (C++17).
- **clangd & clang-tidy:** Normalmente instalados junto com LLVM/Clang. Ubuntu: `sudo apt-get install clangd clang-tidy`.
- **Python 3.10+:** instale via gerenciador do SO.
- **Pip (Python):** após instalar Python, garanta que `pip` esteja disponível.
- **Bibliotecas pip:**

```
pip install scikit-build-core pybind11 pytest ruff
```


- **CodeScene CLI (opcional):** A extensão CodeScene roda no editor, mas você pode instalar a CLI para análise offline: `pip install codescene`.
- **Eigen:** No Linux, normalmente disponível via pacote `sudo apt-get install libeigen3-dev`. (Ou baixe do site da Eigen [\[24†L1-L3\]](#)).
- **(Opcional):** Ferramentas de build como Ninja para acelerar CMake (`sudo apt-get install ninja-build`).

Perfil VS Code e Arquivos de Configuração

- **Perfil VS Code:** crie um perfil específico (Configurações → Profiles → Create Profile “Code Smell Lab”) que inclua:
- **extensions.json** no workspace (ver abaixo) com recomendações e instalação automática (CodeScene, clangd, Python, Live Share, Ruff, GitLens, etc.).
- **settings.json** no workspace para configurações úteis, por exemplo:

```
{
  "python.defaultInterpreterPath": "/usr/bin/python3",
  "python.linting.enabled": false,           // vamos usar Ruff em vez do
  linter integrado
  "ruff.enable": true,
  "ruff.executable": "ruff",
  "clangd.path": "/usr/bin/clangd",
  "clangd.arguments": ["--clang-tidy"],
  "codescene.hostedUrl": "https://codescene.com", // se for usar análise
  remota
  // Demais configurações específicas da aula...
}
```

- **tasks.json:** definindo tarefas de automação. Por exemplo:

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "Build C++",
      "type": "shell",
      "command": "cmake --build build -- -j4",
      "group": "build"
    },
    {
      "label": "Run All Analyses",
      "type": "shell",
      "command": "ruff check . && clang-tidy cpp/solver.cpp -- && pytest --
maxfail=1 --disable-warnings",
      "presentation": { "reveal": "always" }
    }
  ]
}
```

```
]
}
```

Isso permite, por exemplo, `Ctrl+Shift+B` para build, e outra Task (ou atalho) para rodar Ruff, clang-tidy e pytest de uma só vez.

- **launch.json**: não essencial, mas útil se quiser debug de C++ ou Python.
- **.vscode/extensions.json** (workspace): recomendações de extensões para quem abrir o projeto. Exemplo:

```
{
  "recommendations": [
    "astral-sh.ruff",
    "cschleiden.vscode-github-actions",
    "mads-hartmann.codescene-ide",
    "llvm-vs-code-extensions.vscode-clangd",
    "ms-python.python",
    "ms-python.vscode-pylance",
    "ms-vsliveshare.vsliveshare",
    "sharkdyne.gitme"
  ],
  "unwantedRecommendations": []
}
```

Assim, o VS Code sugere a instalação dessas extensões ao abrir o projeto.

Estrutura de Projeto e Exemplo de Build

Suponha estrutura *smell-lab/* no host:

```
smell-lab/
├── cpp/
│   ├── include/
│   │   └── solver.hpp
│   ├── src/
│   │   ├── solver.cpp
│   │   └── bindings.cpp
├── python/
│   ├── mathlab/           # módulo Python para código público
│   │   ├── __init__.py
│   │   └── api.py
├── tests/
│   └── test_solver.py
```

```
├─ CMakeLists.txt
├─ pyproject.toml
└─ smell-lab.code-workspace
```

• **CMakeLists.txt** (exemplo mínimo):

```
cmake_minimum_required(VERSION 3.15)
project(mathlab LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 17)
find_package(pybind11 REQUIRED)
find_package(Eigen3 REQUIRED)

# Cria o módulo _solver (na prática, _mathcore.so por pybind11)
pybind11_add_module(_mathcore cpp/bindings.cpp cpp/src/solver.cpp)
target_include_directories(_mathcore PRIVATE cpp/include)
target_link_libraries(_mathcore PRIVATE Eigen3::Eigen)
```

Aqui assumimos que `solver.cpp` contém as funções matemáticas e `bindings.cpp` declara as funções no módulo `PyInit__mathcore`. O `pybind11_ADD_MODULE` gerará `~/_mathcore*.so`.

• **pyproject.toml** (para scikit-build):

```
[build-system]
requires = ["scikit-build-core", "pybind11"]
build-backend = "scikit_build_core.build"

[project]
name = "mathlab"
version = "0.1.0"
authors = [{name="Equipe Código Limpo"}]
dependencies = ["numpy"]
```

Esse arquivo indica que `scikit-build-core` será usado para criar a extensão em C++. O módulo resultante (`_mathcore.so`) será instalado sob o pacote `mathlab`.

Instale localmente com:

```
pip install -e .
```

Isso executa o CMake e compila o módulo, criando `mathlab/_mathcore*.so`.

Instruções para Alunos

Durante a dinâmica, os alunos **não precisam instalar todo o stack**, pois farão parte de uma sessão Live Share *hosteada pelo professor*. Neste modelo:

- **No computador do aluno:** instale apenas o **VS Code** e a extensão **Live Share** (convidado). (A Microsoft orienta que apenas estes são suficientes para colaboração, pois o host fornece a maior parte do ambiente [31†L41-L48] .)
- **No host (professor):** já instalado tudo listado acima. Os alunos conectam via link do Live Share e veem o código, podendo editar junto (o host pode habilitar/desabilitar edição por aluno para cada arquivo).

Se, por outro lado, quisermos que cada aluno rode localmente (por exemplo, competição entre equipes em máquinas próprias), cada um precisaria replicar o ambiente de análise: instalar Python, pip, pybind11, Eigen, clangd, Ruff e tudo o que o host tem. Neste caso, seria quase como instruir um *setup de pré-aula*, usando instruções similares às do host.

Resumo prático para alunos (modo Live Share):

1. Abrir VS Code.
2. Instalar extensão **Live Share** e fazer login (gratuito).
3. Entrar no link fornecido pelo professor (Normalmente basta clicar no link da sessão).
4. O repositório do exercício aparecerá e as extensões compartilhadas pelo host serão ativadas automaticamente.
5. Seguir o guia de atividades (lido no `README.md` do projeto).

Caso precisem rodar localmente (não via Live Share), o **Student_Guide.md** (abaixo) explicará como instalar rapidamente as ferramentas principais (Python 3, pip, Eigen, clangd, Ruff, CodeScene extensão). Mas idealmente o host compartilha tudo via Live Share.

Scripts e Comandos Práticos

Para manter o fluxo ágil, é bom ter comandos prontos:

- **Build C++ / instalar pacote Python:**

```
pip install --upgrade pip
pip install cmake ninja # necessário para scikit-build/CMake
pip install scikit-build-core pybind11 # para o build
pip install -e . # executa CMake e build
```

- **Executar testes:**

```
pytest tests/ --maxfail=1 --disable-warnings
```

- **Rodar linter static:**

```
clang-tidy cpp/src/solver.cpp -checks=*,bugprone-*,... -- # diagnósticos C++
ruff check python/mathlab/ tests/ --select E,F,B,C90,SIM # códigos python
```

- **Tasks do VS Code** (como configurado acima):
- **Ctrl+Shift+B** para compilar (equivalente a `cmake --build`).
- Outra Task (definida como “Analyze”) roda Ruff + clang-tidy + pytest. Por exemplo:

```
ruff check . && clang-tidy cpp/src/solver.cpp -- && pytest
```

- Ou via paleta de comandos, execute `Tasks: Run Task → Run All Analyses`.

- **Uso do Live Share:**

- Host: abra o projeto, clique no ícone Live Share (barra inferior) e selecione *Start Collaboration Session*. Copie o link e envie aos alunos.
- Alunos: abrir o link no browser ou no VS Code (Menu → Live Share: Join Collaboration Session).
- Dica: professor pode compartilhar o terminal (share server) e até o servidor de testes, permitindo que execução no host apareça para todos `[31†L65-L73]`. Isso facilita apresentar um `pytest` em tempo real para todos os convidados.
- Limitador: se o Live Share cair ou plugins não funcionarem 100%, a alternativa é fazer *screen-share* ou passar o código atual para um canal comum (Git/Gists) e pedir para todos rodarem localmente as Tasks.

Exemplos de Código e Exercícios

1. Código C++ de Exemplo (solver.hpp / solver.cpp)

solver.hpp:

```
#pragma once
#include <Eigen/Dense>

// Função que resolve Ax = b pelo método de eliminação de Gauss
Eigen::VectorXd solveLinear(const Eigen::MatrixXd& A, const Eigen::VectorXd& b);
```

solver.cpp (versão *ruim* inicial):

```
#include "solver.hpp"
#include <Eigen/Dense>
#include <vector>
```

```
// Exemplo de code smells: função grande, muitos comentários, uso ineficiente da
// biblioteca
Eigen::VectorXd solveLinear(const Eigen::MatrixXd& A, const Eigen::VectorXd& b)
{
    // Suponha A é quadrada e invertível
    int n = A.rows();
    Eigen::MatrixXd M = A; // cópia para manipular
    Eigen::VectorXd x(n);
    // Inicializa x
    for (int i = 0; i < n; ++i) {
        x[i] = 0;
    }
    // Eliminação Gaussiana básica (sem pivoteamento)
    for (int k = 0; k < n; ++k) {
        double pivot = M(k,k);
        if (pivot == 0) {
            // Sem solução simples
            return Eigen::VectorXd::Zero(n);
        }
        for (int j = k+1; j < n; ++j) {
            double factor = M(j,k) / pivot;
            for (int i = k; i < n; ++i) {
                M(j,i) = M(j,i) - factor * M(k,i);
            }
            x[j] = x[j] - factor * x[k];
        }
    }
    // Substituição para trás
    for (int i = n-1; i >= 0; --i) {
        double sum = b[i];
        for (int j = i+1; j < n; ++j) {
            sum -= M(i,j) * x[j];
        }
        x[i] = sum / M(i,i);
    }
    return x;
}
```

Neste código C++, inserimos deliberadamente **smells**:

- **Long Method**: a função tem muitos loops aninhados.
- **Deep Nested Complexity**: dois loops aninhados dentro de outro (ciclos duplos).
- **Primitive Obsession**: valores 0 usados diretamente, em vez de constantes nomeadas.
- **Duplication / Ineficiência**: cópia inteira da matriz **A** em **M** (poderia usar ref ou pivot direto).
- **Falha silenciosa**: retorno de vetor zero em caso de pivô 0 (não é ideal; deveria lançar exceção ou outro tratamento).

2. Bindings pybind11 (bindings.cpp)

bindings.cpp:

```
#include <pybind11/pybind11.h>
#include <pybind11/eigen.h>
#include "solver.hpp"

namespace py = pybind11;

PYBIND11_MODULE(_mathcore, m) {
    m.doc() = "Módulo nativo de solver"; // docstring do módulo
    m.def("solve_linear", &solveLinear, "Resolve sistema linear Ax=b");
    // Poderíamos adicionar mais funções, classes, etc.
}
```

Esse arquivo cria um módulo `_mathcore` (nome fictício) onde a função `solveLinear` é exposta como `solve_linear` em Python. O pybind11 lida automaticamente da conversão entre `Eigen::MatrixXd` e `numpy.ndarray` (sem cópia, via buffer) [7†L137-L140] .

3. Código Python de Uso (api.py)

No pacote Python, por exemplo `python/mathlab/api.py` :

```
from . import _mathcore
import numpy as np

def solve_system(A, b):
    """
    Wrapper Python amigável: converte inputs em numpy e chama a função C++.
    """
    A = np.array(A, dtype=float)
    b = np.array(b, dtype=float)
    if A.ndim != 2 or b.ndim != 1:
        raise ValueError("Formato inválido para A ou b")
    # Chama o solver C++
    return _mathcore.solve_linear(A, b)
```

O usuário Python importa apenas `solve_system` ; internamente, vai para o C++ otimizado. A camada Python faz validações básicas.

Uso no Python:

```

from mathlab.api import solve_system

A = [[2,1],[1,3]]
b = [4,7]
x = solve_system(A, b)
print(x)  # [1, 2] (solução: x = [1,2])

```

4. Testes Pytest

Um exemplo simples em `tests/test_solver.py`:

```

import numpy as np
from mathlab.api import solve_system

def test_solve_identity():
    A = np.eye(3)
    b = np.array([1,2,3], dtype=float)
    x = solve_system(A, b)
    assert np.allclose(x, b)

def test_solve_known():
    A = np.array([[3.0,1.0],[0,2.0]])
    b = np.array([5.0, 4.0])
    x = solve_system(A, b)
    # Espera x = [1,2]
    assert np.allclose(x, np.array([1.0, 2.0]), atol=1e-6)

def test_non_square():
    A = np.array([[1.0, 2.0, 3.0]])
    b = np.array([4.0])
    try:
        solve_system(A, b)
        assert False, "Deveria levantar erro ValueError"
    except ValueError:
        pass

```

Isso garante que a refatoração não quebre casos simples. Se qualquer teste falhar, sabemos que o comportamento mudou incorretamente.

5. Code Smells Introduzidos (intencionais)

No código acima, podemos apontar:

- **C++**: método longo, loops aninhados, lógica de pivotamento minimalista. *CodeScene* identificaria possivelmente “Deep Nested Complexity” e “Complex Method” [\[36†L225-L234\]](#) . O clang-tidy

provavelmente avisará de estilo e possibilidades de otimização (como `-checks=performance-*` ou `modernize-*`).

- **Python:** a camada `api.py` está limpa, mas no exercício podemos criar outro exemplo Python “ruim”. Por exemplo:

```
def process_items(items):
    result = []
    for item in items:
        if item.active:
            if item.value > 0:
                if item.value < 100:
                    result.append(item)
    return result
```

Esse código tem *nested ifs* aninhados desnecessariamente. O **Ruff** sinalizaria complexidade (`C901`) e simplificações (`SIM`), e o CodeScene sinalizaria “Complex Conditional”. (Os alunos poderiam refatorar usando *guard clauses* ou lógica booleana simples: `if item.active and 0 < item.value < 100:`.)

- **Integração:** podemos imaginar que a assinatura exposta do solver tem muitos parâmetros (flags) em vez de usar estruturas. P. ex., mudar `solveLinear` para receber um objeto `SolverOptions` (evita *Primitive Obsession*). No Python, evitar passar três vetores separados, agrupando em objetos. Esses são conceitos de design de API.

6. Exercícios de Refatoração

Alguns exercícios possíveis:

1. **Extrair Método:** no `solveLinear`, uma parte da eliminação poderia virar função separada (por ex. divisão do sistema, back-substitution), tornando o método principal menor.
2. **Substituir Cópia Inútil:** remover cópia `Eigen::MatrixXd M = A` e operar diretamente em `A` (ou usar `Eigen::Ref`).
3. **Tratar Erro de Pivot:** em vez de retornar vetor zero silenciosamente, lançar `std::runtime_error` e capturá-lo no Python (mais seguro).
4. **API Mais Clara:** se tivéssemos vários parâmetros, agrupar em `struct` ou argumentos nomeados (no Python, converter *args para *kwargs* com valores default).
5. **Python Conditional Simple:** para o exemplo de *nested if* em Python, refatorar para uma única condição composta ou retornar cedo.

Critérios de aceitação:

- Todos os **testes passam** (não alteramos a lógica final, apenas forma).
- O **Code Health** (pontuação CodeScene) dos arquivos refatorados aumenta (indicando código mais legível/manutenível) [36†L225-L234] .
- O número de **warnings** (clang-tidy e Ruff) diminui ou mantém apenas o estritamente necessário.

Por exemplo, extrair método deve manter `pytest` passando e fazer o CodeScene reportar método menor. O uso de guard clauses ou operadores lógicos evita aninhamentos profundos, eliminando avisos de complexidade (`SIM` no Ruff).

Documentação e Arquivos Markdown

A seguir estão os conteúdos dos arquivos `.md` preparados para a atividade. Cada um pode ser salvo diretamente no repositório:

README.md

Code Smell Lab

****Objetivo:**** Refatorar código legado com *code smells* usando C++ ↔ Python.

Neste laboratório, integraremos um núcleo numérico em C++ com Python. Faremos *pair programming* em VS Code, usando Live Share para colaborar. Ferramentas como clang-tidy, Ruff e CodeScene mostrarão problemas em tempo real.

****Passos principais:****

1. Analisar o código atual e identificar *code smells*.
2. Refatorar preservando o comportamento (tests Python devem continuar passando).
3. Verificar melhorias nas métricas (Code Health ↑, warnings ↓).

****Agenda:****

- Introdução a code smells e refactoring.
- Demonstração de ferramentas (CodeScene, clangd, Ruff).
- Prática em equipes: refatorar código *legacy* (``cpp/`` e ``python/``).
- Checkpoints: medir resultados e discutir.
- Live Share: demonstração de refactor em tempo real.

****Outros arquivos úteis:****

- [``HOST_SETUP.md``](HOST_SETUP.md): Instruções de configuração para o professor/host.
- [``STUDENT_GUIDE.md``](STUDENT_GUIDE.md): Guia rápido para alunos (instalações mínimas).
- [``EXERCISES.md``](EXERCISES.md): Detalhes dos exercícios e critérios de avaliação.

****Ferramentas principais:****

- VS Code com extensões: Live Share, CodeScene, LLVM/clangd, Ruff, Python.
- Pybind11 (C++ ↔ Python), Eigen (área linear), scikit-build-core (build).
- clang-tidy (C++ analysis), Ruff (Python lint), pytest (testes).

Vamos codar com qualidade! ;)

HOST_SETUP.md

Configuração do Ambiente (Host)

Estas instruções são para preparar o computador do ****professor/host**** antes da aula.

Instalações Obrigatórias

- ****VS Code****: Instale a versão mais recente.
- ****Extensões VS Code**** (você pode criar um profile para importar depois):
 - Live Share
 - CodeScene IDE Extension
 - C/C++ (Microsoft) e clangd LSP (LLVM)
 - Ruff (Astral)
 - Python (Microsoft)
- ****Compilador C++****: g++ 11+ ou clang++ 11+.
- ****Python 3.10+****: instale via pacote do sistema ou python.org.
- ****pip**** (gerenciador de pacotes Python).

Pacotes pip (Python)

No terminal, execute:

```
pip install --upgrade pip pip install scikit-build-core pybind11 pytest ruff codescene
```

- `scikit-build-core` (integra CMake no `pip install`).
- `pybind11` (libraries de bindings).
- `pytest` (testes).
- `ruff` (linter rápido).
- `codescene` (CLI opcional).

Biblioteca C++ Eigen

- Linux: `sudo apt-get install libeigen3-dev`
- macOS (Homebrew): `brew install eigen`
- Alternativa: baixe do [Eigen gitlab](https://gitlab.com/libeigen/eigen/) e extraia.

Verificar Instalações

- `clangd --version`, `clang-tidy --version` devem funcionar.
- `python3 --version` deve indicar ≥ 3.10 .
- `pip list` mostra **scikit-build-core**, **pybind11**, **pytest**, **ruff**, **codescene** instalados.

Configuração do Projeto

1. Clone o repositório / crie pasta `smell-lab/` com a estrutura fornecida.
2. Abra no VS Code e importe as extensões recomendadas (`vscode/extensions.json`).
3. Certifique-se de que o `CMakeLists.txt` e `pyproject.toml` estão no root do projeto.
4. Teste o build inicial:


```
```bash
```

```
mkdir build && cd build
cmake .. -G Ninja
ninja
```

```
```
```

5. Se usar scikit-build, apenas:


```
```bash
```

```
pip install -e .
```

```
```
```

6. Execute `pytest` e verifique que todos os testes passam antes da aula.

Agora você está pronto para compartilhar via Live Share e receber os alunos.

STUDENT_GUIDE.md

Guia Rápido para Alunos

Antes da Aula (somente se solicitado)

Caso participem **fora da sessão Live Share**, cada aluno precisará de:

- **VS Code** instalado.
- Extensão **Live Share** (para entrar na sessão do professor).
- (Opcional) Se forem rodar localmente, instalem também: **clangd**, **clang-tidy** e **Ruff**.
 - Linux: `sudo apt-get install clangd clang-tidy`
 - Ruff: `pip install ruff`.
 - Eigen (Dev C++): conforme instruções do host.
 - Python 3.10+ e `pip` (para rodar testes).

- (Opcional) Instalar o pacote do exercício:

```
```bash
```

```
git clone <repositório>
cd smell-lab
pip install -e .
```

```
```
```

- ****Não é necessário**** instalar CodeScene (o professor já terá na sessão) nem configurar Tasks do VS Code (faremos na aula).

Durante a Aula

1. Abra o VS Code.
2. Instale ****somente**** a extensão Live Share e faça login (Microsoft ou GitHub).
3. Clique no link de convite do professor para entrar na sessão colaborativa.
4. No editor compartilhado, você verá o código sendo editado pelo grupo.
5. Siga as instruções do professor:
 - Use o terminal compartilhado para compilar ou rodar testes, se solicitado.
 - Observe as marcações de CodeScene (painel "Problems" aponta code smells).
 - Corrija o código em pares: o professor apontará qual time deve editar em cada etapa.
6. Não se esqueça de executar ``pytest`` localmente (ou via Task) para garantir que o código ainda funciona após alterações.
7. No final, cada time mostrará suas melhorias. Fique atento às métricas no CodeScene (indicador de "Code Health").

Dicas Úteis

- Use ``Ctrl+Shift+B`` (ou menu Run → Run Build Task) para compilar tudo rapidamente.
- Abra o **Problems Panel** do VS Code (ícone com ponto de exclamação) para ver alertas do CodeScene, clangd e Ruff.
- Para aplicar **quick fixes** do clang-tidy: quando um warning aparece, passe o mouse e clique na lâmpada ou use o menu de context.
- O CodeScene mostra códigos afetados por **code smells** em tempo real: veja a pontuação "Code Health" no topo do arquivo.
- Se o Live Share estiver instável, foque em discussões de refactoring enquanto o professor reestabelece a sessão.

Boa prática e participação! Ajuste ideias em grupo, sempre preservando os testes.

EXERCISES.md

Exercícios e Critérios de Aceitação

Desafios (para cada time)

Os times terão que refatorar o código legado de forma iterativa. Exemplos de desafios:

1. **Solver C++**: melhorar a função `solveLinear`:
 - Extrair partes em sub-funções (ex.: matriz triangular, back-substitution).
 - Evitar cópias desnecessárias de matrizes.
 - Tratar possíveis erros (lançar exceções em vez de resultados silenciosos).
2. **Integração Python**: aprimorar a API `solve_system`:
 - Validar entradas (já implementado no exemplo).
 - Se houvesse muitos parâmetros de configuração, reduzir usando classes/structs.
3. **Código Python adicional**: (exemplo livre)
 - Transformar condicionais aninhadas em expressões simples.
 - Aplicar *guard clause* (early return) para evitar nesting.
 - Remover duplicações de lógica em loops, etc.

Critérios de Aceitação

- **Comportamento preservado**: Todos os *pytest* devem passar após a refatoração.
- **Testes mantidos**: Não modifique ou exclua testes para atingir métricas; os testes definem o contrato.
- **Código mais limpo**: O *Code Health* do arquivo refatorado deve aumentar (CodeScene registra automaticamente).
- **Menos warnings**: Warnings de *clang-tidy* ou *Ruff* devem reduzir (ou ao menos não aumentar).
- **Critérios Específicos**: Cada exercício (listado acima) pode ter metas próprias, p. ex. “extrair método X, nomear variável Y adequadamente”.

Exemplo de Critério por Exercício

- *_Challenge “Eliminação Gaussiana”_*: Pode requerer reduzir a complexidade ciclomática em 30%.
- *_Challenge “Python Nested If”_*: Consolidar as 3 condições em um único `if` ou `guard clause`.
- *_Challenge “API Design”_*: Não utilizar mais de 4 parâmetros básicos em chamada de função; usar `struct` se necessário.

> ****Nota:**** A ideia não é apenas “fazer os erros sumirem”, mas melhorar a estrutura. Comparar antes e depois: a pontuação “Code Health” mostrará se a legibilidade/mantenibilidade evoluiu.

Boa sorte, e mãos ao código!

Diagramas de Auxílio

Além do cronograma acima, um diagrama simplificado da arquitetura ajuda a visualizar a integração **Python ↔ C++**:

```
flowchart LR
    subgraph Python_App
        direction LR
        A1[Notebook / Script Python] -->|`import matlab.api`| A2[matlab/_mathcore.so]
    end
    subgraph Native_Module
        direction LR
        A2 -->|pybind11 binding| B1[C++ Solver (Eigen)]
        B1 --> B2[Eigen::MatrixXd, Eigen::VectorXd]
        B2 --> B1
        B1 -->|resultados| A2
    end
    A2 --> A1
    style A1 fill:#f9f,stroke:#333,stroke-width:1px
    style B1 fill:#cff,stroke:#333,stroke-width:1px
```

Na prática, o **Python chama o módulo nativo** `_mathcore.so` (gera automaticamente via CMake+pybind11). Este módulo contém as funções C++, usando o Eigen para cálculos. O fluxo é: Python passa um `numpy.ndarray` → `pybind11` converte para `Eigen::MatrixXd` por referência → C++ executa algoritmos → retorna `Eigen::VectorXd` → `pybind11` converte de volta para `numpy.ndarray`. Dessa forma, o Python “vê” apenas uma função nativa rápida, e todo o pesado fica no C++.

Referências:

- Pybind11 (C++ ↔ Python) [【7†L59-L64】](#) , nanobind benchmarks [【4†L58-L66】](#) , cppy vs pybind11 [【4†L130-L138】](#) .
- Criação de pacote Python com CMake (scikit-build-core) [【23†L95-L102】](#) .
- Ruff (linter Python ultrarrápido) [【29†L57-L60】](#) .
- clang-tidy (análise estática C++) [【26†L32-L36】](#) .
- CodeScene (Code Health, *code smells*) [【36†L225-L234】](#) [【36†L268-L274】](#) .
- VS Live Share (colaboração) [【31†L41-L48】](#) .
- gRPC (RPC de alto desempenho) [【20†L17-L22】](#) , ZeroMQ (mensageria IPC) [【16†L47-L50】](#) .

Este relatório é completo e foi elaborado com base em fontes oficiais e documentação atualizada (2024/2025). Ele está pronto para ser usado como guia de implementação do workshop de refatoração e code smells. Boa aula!
